AWS CLOUD INFRASTRUCTURE & SYSTEMS LAB

Amazon EBS & S3 Architecture and Volume Management Lab

A practical reference manual and execution guide for block storage configuration, filesystem initialization, file handling, and volume detachment workflows.

---

1. Architectural Comparison: Amazon EBS vs. Amazon S3

Amazon Elastic Block Store (EBS) and Amazon Simple Storage Service (S3) serve fundamentally different storage architectures within AWS cloud deployments:

Dimension	Amazon EBS (Elastic Block Store)	Amazon S3 (Simple Storage Service)
Storage Paradigm & Data Type	**Block Storage:** Data is organized into raw, unformatted blocks with no inherent metadata structure. Operates like a physical external disk attached via high-speed SAN.	**Object Storage:** Data is stored as discrete objects containing payload (data), a unique key (URL/identifier), and rich custom metadata tags.
Access Method & Protocol	Attached directly to a running EC2 instance via NVMe/Xen block device drivers. Formatted with local filesystems (ext4, XFS) and accessed via POSIX I/O calls.	Decoupled web-scale access over HTTP/HTTPS REST APIs. Accessible globally via AWS SDKs, CLI, or direct web URLs without mounting.
Scalability & Availability	Bound to a single Availability Zone (AZ). Volume sizes expand dynamically up to 64 TiB, with IOPS provisioned per volume tier. Replicated within 1 AZ.	Serverless, virtually infinite scalability across multiple AZs automatically. Delivers 99.999999999% (11 9s) of data durability.
Typical Use Cases	OS root boot drives, relational/transactional database engines (PostgreSQL, MySQL, Oracle), high-throughput transactional logging.	Static web assets, user image/video uploads, data lake raw storage, disaster recovery backups, big data analytics pipelines.
Real-World App Example	**Zomato High-Speed Transaction DB:** PostgreSQL/MySQL database instance hosting active restaurant tables, order fulfillment state, and live rider checkout updates where sub-millisecond block latency is critical.	**Zomato Restaurant Menu & Food Photos:** Storing millions of user-uploaded food images, dish photographs, and digitized menu cards served directly to customer mobile apps via CloudFront CDN.

2. EC2 Free Tier Provisioning & EBS Volume Attachment

Follow these operational steps using the AWS Management Console to set up the host compute and block storage resource:

1. **Launch EC2 Instance:**
   - Navigate to **EC2 Console > Instances > Launch Instances**.
   - Select **Amazon Linux 2023 AMI** (Free tier eligible) and the **t2.micro** (or **t3.micro** depending on region) instance type.
   - Configure or create an active Key Pair (`.pem`), assign an active security group permitting SSH traffic (port 22), and click **Launch Instance**.
   - *Crucial Note:* Note the **Availability Zone** (e.g., `ap-south-1a`) where the EC2 instance is placed.

2. **Create EBS Volume:**
   - Navigate to **Elastic Block Store > Volumes > Create Volume**.
   - Select **General Purpose SSD (gp3)** and allocate a size of at least **2 GiB**.
   - **Zone Matching:** You must select the *exact same Availability Zone* as your launched EC2 instance; EBS volumes cannot bridge AZs directly.
3. **Attach Volume to Instance:**
   - Select the newly created 2 GiB volume, click **Actions > Attach volume**.
   - Search for your running instance ID, choose the default device assignment (e.g., `/dev/xvdf` or `/dev/sdf`), and confirm attachment.

3. SSH Authentication, Filesystem Formatting & Directory Mounting

Connect to the instance via your terminal using your private key and execute the initial partition configuration:

```
# Connect to your Amazon Linux EC2 instance ssh -i /path/to/your-key.pem ec2-user@<EC2_PUBLIC_IPv4_ADDRESS> # Inspect available block devices to identify the newly attached 2 GiB disk lsblk
```

In modern Nitro/KVM-based instances, the newly attached drive will appear under NVMe naming conventions (such as `nvme1n1` or `xvdf`).

```
# Verify the volume does not already contain an active filesystem sudo file -s /dev/nvme1n1 # Output should return: "/dev/nvme1n1: data" # Format the volume with an ext4 filesystem sudo mkfs -t ext4 /dev/nvme1n1 # Create the designated target mount point directory sudo mkdir -p /mnt/mydata # Mount the formatted block device to the directory sudo mount /dev/nvme1n1 /mnt/mydata # Validate the mount status, capacity, and path df -hT /mnt/mydata
```

4. Simulating App File Creation & Ingestion (Zomato Pattern)

Simulate an application storing a digitized catalog item on the server and transferring it into the persistent mount:

```
# Create the sample catalog file in the user's home directory echo -e "=== Zomato Restaurant Menu ===\nDish: Butter Chicken | Price: INR 350\nDish: Garlic Naan | Price: INR 45\nStatus: Available" > ~/zomato_menu.txt # Inspect file contents locally cat ~/zomato_menu.txt # Copy the file to the mounted persistent EBS storage volume sudo cp ~/zomato_menu.txt /mnt/mydata/zomato_menu.txt # Confirm file presence, size, and integrity on the mounted volume ls -la /mnt/mydata/ cat /mnt/mydata/zomato_menu.txt
```

Verification Check: The file `zomato_menu.txt` is now written directly into the block blocks of the 2 GiB EBS volume. Any modifications performed at `/mnt/mydata/` persist independently of the host EC2 operating system memory.

5. Volume Detachment, Reattachment & State Persistence Analysis

Execution Procedure

1. Unmount the device cleanly from the operating system to flush kernel cache and prevent corruption:

```
sudo umount /mnt/mydata
```

2. In the **AWS Management Console**, go to **EC2 > Volumes**. Select the 2 GiB volume, choose **Actions > Detach volume**, and confirm.
3. Reattach the volume to the original instance (or a second instance in the same AZ) by selecting **Actions > Attach volume**.
4. Back in the SSH terminal, re-mount the drive directly (**do not run mkfs again**):

```
sudo mount /dev/nvme1n1 /mnt/mydata cat /mnt/mydata/zomato_menu.txt
```

Behavior of Stored Data

All data stored on the volume (`zomato_menu.txt`) remains **100% intact and undamaged**. Because EBS is an external networked block storage device decoupled from EC2 compute hardware, detaching the volume does not wipe or reset the filesystem blocks.

Industrial Significance (Flipkart, Myntra & E-Commerce Order Tracking)

For mission-critical e-commerce platforms like Flipkart and Myntra, this decoupling behavior is vital for high availability and failure recovery:

- **Crash Recovery & Fault Tolerance:** If an EC2 instance hosting an order fulfillment tracking engine experiences hardware failure or kernel panic, operators can terminate the damaged compute node, spin up a clean instance, attach the existing EBS volume, and resume tracking state without losing ongoing customer deliveries.
- **Zero-Data-Loss Maintenance:** Compute instances can be patched, upgraded to larger CPU/RAM sizes, or swapped across versions while keeping transactional records and persistent application logs fully preserved on the detached block volume.
- **Rapid Disaster Recovery via Snapshots:** Detached and live EBS volumes can be snapshotted to S3 asynchronously, giving platforms point-in-time state backups for flash sales, festive sale peaks, and auditing.